

Angular Moderno

15 - Guards e Segurança de Rotas

2026

SENAI - CRTI



15 - Guards e Segurança de Rotas

Proteção de rotas e simulação de login em aplicações Angular modernas com Standalone Components. Aprenda como o Router decide quem pode e quem não pode acessar cada parte da sua aplicação.

Contexto do Projeto

Nosso projeto de e-commerce evoluiu bastante ao longo das aulas anteriores. Agora chegamos a um ponto crítico: qualquer usuário consegue acessar qualquer rota livremente.

O que já construímos

- Navegação com Router
- Múltiplas páginas: Home, Produtos, Carrinho
- Lazy Loading com `loadComponent`
- Estrutura organizada por features

Novo problema a resolver

Atualmente, não existe nenhum controle sobre quem pode acessar cada rota. Qualquer pessoa, autenticada ou não, navega livremente pela aplicação. Precisamos mudar isso.

⚠ Qualquer usuário pode acessar qualquer rota da aplicação.

Problema Atual

Sem proteção de rotas, a aplicação fica vulnerável. Veja o impacto real disso no contexto de um e-commerce:

Acesso Irrestrito

A rota `/carrinho` pode ser acessada por qualquer pessoa, mesmo sem estar autenticada.

Dados Sensíveis Expostos

Informações de pedidos, endereços e dados do usuário ficam visíveis sem controle de acesso. Em aplicações reais isso não pode acontecer.

Falta de Segurança

Uma aplicação real precisa saber quem está logado antes de liberar acesso a áreas restritas.



Objetivo da Aula

Ao final desta aula, você terá compreendido e implementado os mecanismos fundamentais de proteção de rotas no Angular. O foco é no **funcionamento do Guard**, não em autenticação real com backend.

1

Proteger Rotas com Guards

Aplicar `canActivate` para controlar o acesso a rotas específicas da aplicação.

2

Simular Autenticação

Usar um estado simples com `signal` para representar o login do usuário.

3

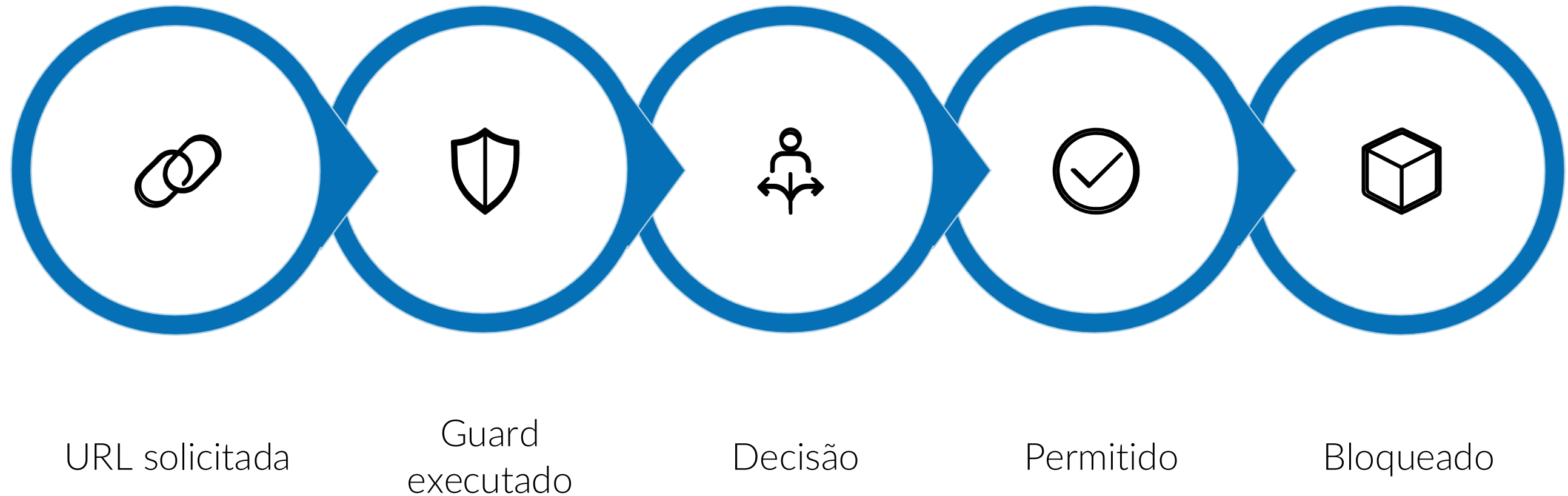
Entender o Fluxo do Router

Compreender como o Angular Router decide permitir ou bloquear o acesso a uma rota.

 O foco desta aula **não** é autenticação real. O foco é entender o funcionamento do Guard e do Router.

Conceito Principal

O Angular Router tem um papel muito mais amplo do que simplesmente renderizar componentes. Ele também é o responsável por decidir **se o usuário pode ou não acessar uma rota**.



Quando o Guard retorna `false`, a navegação é interrompida antes mesmo do componente ser carregado, protegendo tanto a interface quanto os dados.

O Que é um Guard?

Um Guard é uma **função que o Router executa antes de concluir a navegação**. Ele age como um porteiro: verifica as condições e decide se o acesso será liberado ou negado.

Retorna **true**

A navegação é permitida e o componente da rota é renderizado normalmente.

Retorna **false**

A navegação é bloqueada. O usuário permanece na rota atual sem acessar o componente protegido.

No Angular moderno, utilizamos **Guards funcionais**. São simples funções TypeScript, em vez de classes com interfaces. Isso simplifica muito o código.



Estratégia Didática da Aula

Para que o aprendizado seja progressivo e focado, esta aula **isola apenas o comportamento do Guard**. Tecnologias de autenticação real serão introduzidas nas próximas aulas.

O que não usamos nesta aula

- Backend ou API externa
- JWT (JSON Web Token)
- HTTP Interceptors
- Serviços de autenticação complexos

O que usamos nesta aula

- Um estado simples de login (`boolean`)
- Signal do Angular para reatividade
- Um Guard funcional puro

📌 Essa abordagem é intencional: entender o Guard sem ruído de outras tecnologias é a forma mais eficaz de aprender.

Criando o Estado de Login

PASSO 1 – AUTH.TS

O primeiro passo é criar um estado global simples que representa se o usuário está ou não autenticado. Usamos o `signal` do Angular para isso. Ele é reativo e funciona perfeitamente com Standalone Components.

Em `SRC/APP/CORE`, crie um arquivo chamado `auth.ts`

```
import { signal } from '@angular/core';  
  
export const usuarioLogado = signal(false);
```

`signal(false)`

Valor inicial: usuário **não logado**. O tipo inferido é `boolean`.

Reatividade automática

Qualquer componente que leia o `signal` será atualizado automaticamente ao alterar o valor.

Estado global

Por ser exportado de um arquivo separado, pode ser compartilhado por toda a aplicação.

Funções de Login e Logout

PASSO 2 – AUTH.TS

Ainda no arquivo `core/auth.ts`, criamos duas funções simples que alteram o valor do signal. Essas funções representam as ações de autenticação da aplicação.

```
// core/auth.ts (continuação)

export function login() {
  usuarioLogado.set(true);
}

export function logout() {
  usuarioLogado.set(false);
}
```

login() – Define o signal como **true**

Representa o momento em que o usuário se autentica na aplicação. A partir daí, o Guard libera o acesso às rotas protegidas.

logout() – Define o signal como **false**

Encerra a sessão do usuário. As rotas protegidas voltam a ser bloqueadas imediatamente.

Criando o Guard

PASSO 3 – AUTH.GUARD.TS

Com o estado pronto, criamos o Guard funcional. Ele importa o signal e simplesmente retorna seu valor atual — que é exatamente o `boolean` que o Router precisa.

Em `SRC/APP/CORE`, crie um arquivo chamado `auth.guard.ts`

```
import { CanActivateFn } from '@angular/router';
import { usuarioLogado } from '../auth';

export const authGuard: CanActivateFn = () => {
  return usuarioLogado();
};
```

`CanActivateFn`

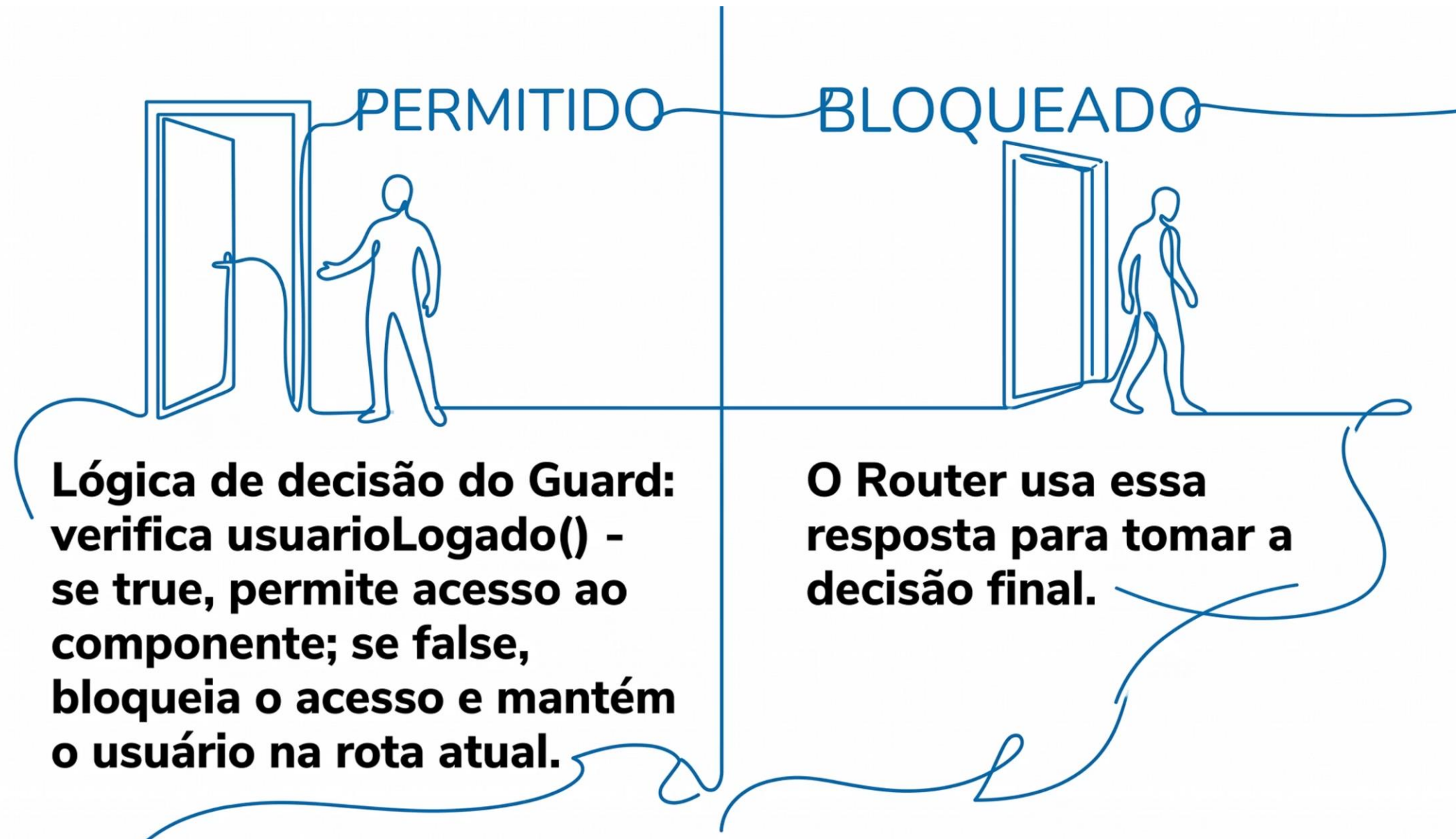
Tipo fornecido pelo Angular que define a assinatura de um Guard funcional moderno.

`usuarioLogado()`

A chamada com `()` lê o valor atual do signal, retornando `true` ou `false`.

Entendendo o Guard

O Guard é executado **antes da navegação ser concluída**. Ele funciona como uma verificação intermediária entre a solicitação de rota e a renderização do componente. **O Guard NÃO renderiza nada. Ele apenas decide.**



Protegendo a Rota

Com o Guard criado, basta associá-lo à rota desejada usando a propriedade `canActivate` no arquivo de rotas.

A rota `/carrinho` agora exige autenticação. Edite o arquivo `app.routes.ts`, conforme os destques abaixo:

```
import { authGuard } from './core/auth.guard';
```

PASSO 4 – APP.ROUTES.TS

```
...
```

```
{  
  path: 'carrinho',  
  canActivate: [authGuard],  
  loadComponent: () => import('./features/carrinho/carrinho/carrinho').then(m => m.Carrinho),  
},
```

PASSO 5 – APP.ROUTES.TS

→ `canActivate: [authGuard]`

Registra o Guard na rota. O Angular executa cada Guard da lista em sequência antes de permitir o acesso.

→ Lazy Loading mantido

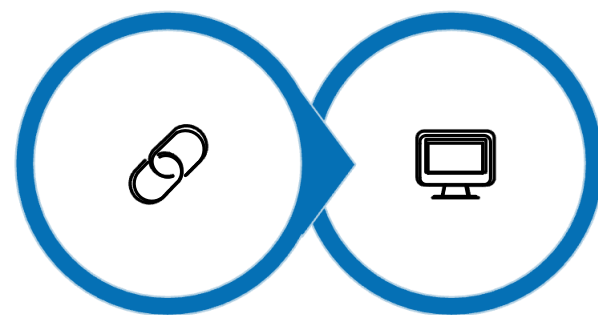
O `loadComponent` continua funcionando normalmente. O Guard é executado antes mesmo do chunk ser carregado.

O Que Mudou no Router

A adição do Guard representa uma mudança conceitual importante no papel do Angular Router dentro da aplicação.

Antes — Rota sem proteção

O Router recebia a URL e imediatamente renderizava o componente correspondente, sem nenhuma verificação de segurança.



URL solicitada

Componente
renderizado

Agora — Rota com Guard

O Router agora inclui uma etapa de verificação. O componente só é renderizado se o Guard aprovar o acesso.



URL solicitada

Guard
executado

Componente
renderizado

Integração com a Interface

PASSO 6 – APP.TS

Para que os botões de Login e Logout funcionem na view, o componente principal (App) precisa importar e expor o estado e as funções do arquivo `auth.ts`. **Deixe o arquivo `app.ts` exatamente assim:**

```
import { Component } from '@angular/core';
import { RouterOutlet, RouterLink } from '@angular/router';
import { usuarioLogado, login, logout } from '../core/auth';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, RouterLink],
  templateUrl: './app.html',
  styleUrls: ['./app.css'],
})
export class App {
  usuarioLogado = usuarioLogado;
  login = login;
  logout = logout;
}
```

-  Como o signal e as funções são exportados diretamente do arquivo `auth.ts`, não precisamos de um serviço injetável. Isso simplifica a implementação neste estágio didático.

Ao expor `usuarioLogado` como propriedade da classe, o template consegue lê-lo de forma reativa, atualizando a UI automaticamente quando o estado muda.

Interface de Login Simples

PASSO 7 – APP.HTML

O template do componente principal exibe dois botões para simular as ações de autenticação e um indicador de status que reage ao signal em tempo real. Em `app.html`, insira estes botões antes de `<router-outlet></router-outlet>`

```
<!-- app.html -->
<button (click)="login()">Login</button>
<button (click)="logout()">Logout</button>

<p>Status: {{ usuarioLogado() ? 'Logado' : 'Deslogado' }}</p>
```



Botão Login

Chama a função `login()`, alterando o signal para `true`. O status na tela atualiza instantaneamente.



Botão Logout

Chama a função `logout()`, alterando o signal para `false`. O acesso a rotas protegidas é revogado.



Indicador de Status

Exibe o estado atual via interpolação reativa. Reflete sempre o valor mais recente do signal.



O signal torna a interface reativa automaticamente

Estrutura Final do App

Com todos os elementos integrados, a aplicação agora possui uma arquitetura completa com navegação, segurança e renderização dinâmica.



O `<router-outlet>` continua sendo o ponto central de renderização. Ele exibe o componente autorizado pelo Guard ou o usuário permanece na rota atual quando o acesso é bloqueado. O Guard é executado antes do carregamento do componente (inclusive no lazy loading)

Teste 1 — Sem Login

Com a aplicação rodando e o usuário **deslogado** (estado inicial), tente acessar diretamente a rota protegida.

Ação

Acesse `/carrinho` sem clicar em Login. Pode ser via link no menu ou digitando a URL diretamente no navegador.



Resultado Esperado

Acesso **bloqueado**. O Guard executa `usuarioLogado()` que retorna `false`, e o Router cancela a navegação.

- ⊗ Se o componente do carrinho aparecer, verifique se o Guard foi corretamente associado à rota com `canActivate: [authGuard]`.

Teste 2 – Com Login

Agora simule o login e tente acessar a rota protegida novamente. O comportamento deve ser completamente diferente.

Clique em "Login"

O signal `usuarioLogado` muda para `true`. O status na interface atualiza para "Logado".

Acesse `/carrinho`

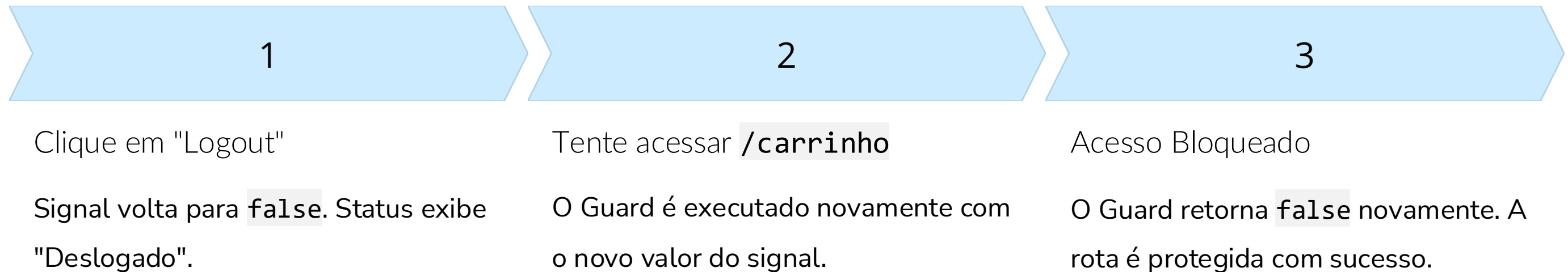
O Router dispara a execução do Guard. Desta vez, `usuarioLogado()` retorna `true`.

Acesso Permitido

O componente `Carrinho` é renderizado normalmente no `router-outlet`.

Teste 3 — Após Logout

O terceiro teste valida que a revogação de acesso funciona corretamente — mesmo após o usuário já ter acessado a rota protegida.



👉 Se os três testes funcionaram conforme esperado, o Guard está implementado corretamente. Parabéns!

Limitação Atual — E Isso é Intencional

Neste momento, a implementação é propositalmente simples. Entender essas limitações faz parte do processo de aprendizado progressivo.

Sem backend ou API

O estado de login existe apenas em memória. Ao recarregar a página, o usuário volta a estar deslogado.

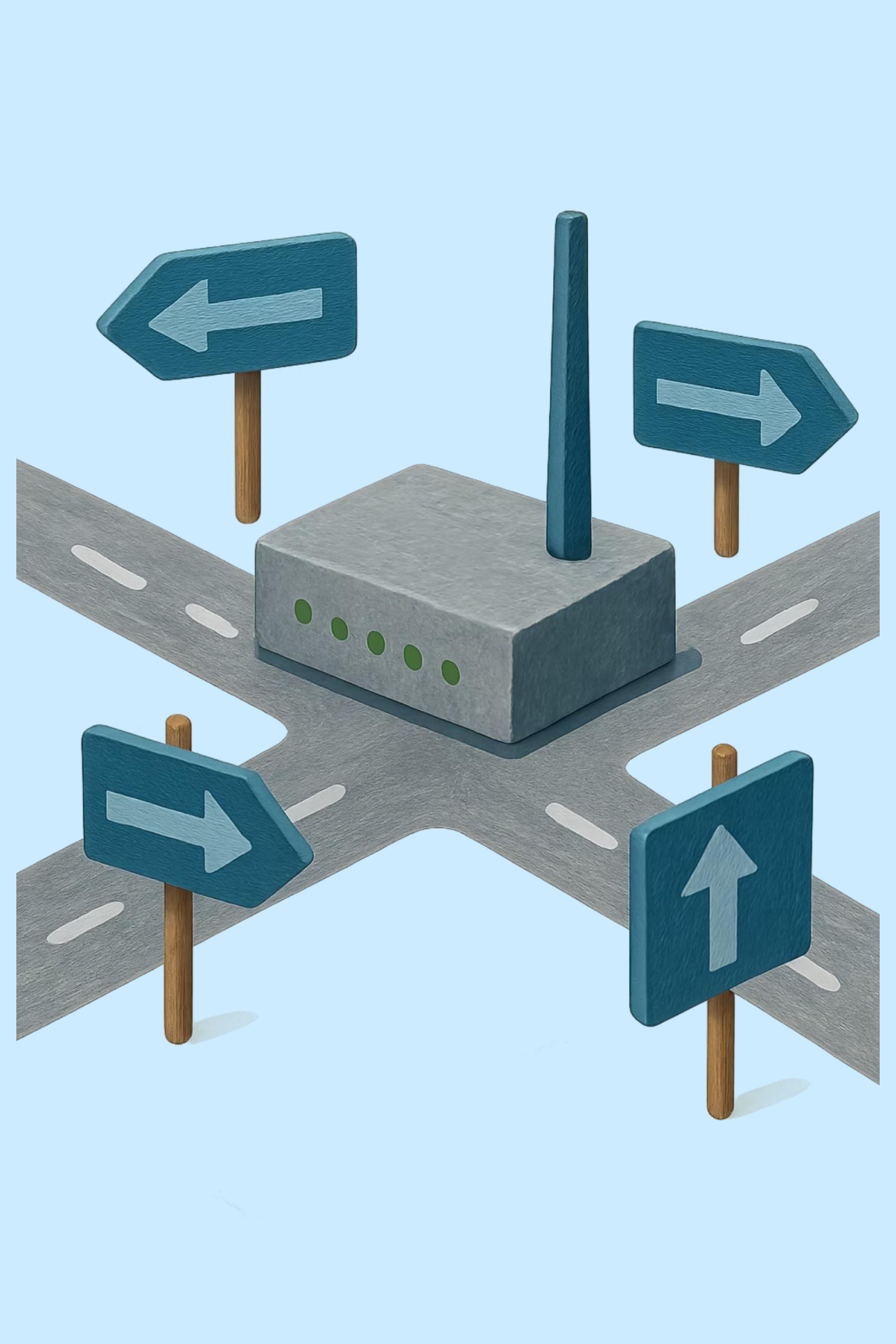
Sem token de autenticação

Não há validação de credenciais. Qualquer clique em "Login" autentica o usuário — isso não é seguro para produção.

Sem redirecionamento

Quando o acesso é bloqueado, o usuário simplesmente fica parado — não é redirecionado para uma tela de login.

- ☐ Todas essas limitações serão resolvidas nas próximas aulas. O foco agora é **entender o Guard**, não construir um sistema completo.



Conceito Crítico

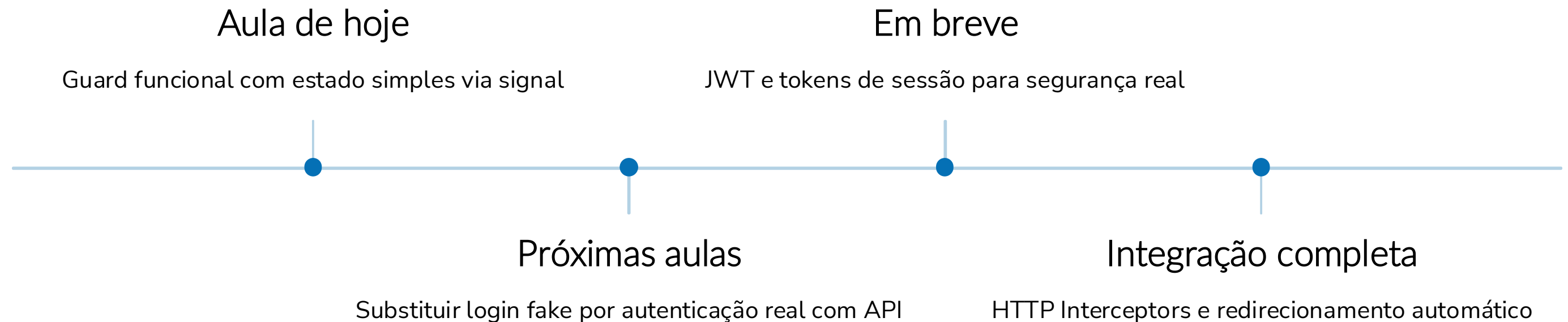
Componentes não controlam acesso.

O Router controla acesso.

Essa é uma das bases mais importantes da arquitetura Angular. Um componente não deve verificar se o usuário está logado para decidir se renderiza ou não. Essa responsabilidade pertence ao Router que, com o Guard, toma essa decisão **antes** do componente sequer ser instanciado.

Conexão com as Próximas Aulas

O que você aprendeu hoje é a fundação. As próximas aulas vão evoluir cada aspecto dessa implementação, tornando a autenticação robusta e pronta para produção.



Tudo isso já está previsto no curso — e agora você tem a base conceitual necessária para absorver cada passo com profundidade.

Resultado da Aula

Ao completar esta aula, você implementou do zero a primeira camada de segurança de uma aplicação Angular moderna. Veja o que está funcionando:



Rotas Protegidas

A rota `/carrinho` agora exige autenticação para ser acessada.



Guard Funcional

Um Guard moderno usando `CanActivateFn` — sem classes, sem injeção complexa.



Simulação de Login

Estado reativo com `signal` e funções simples de `login()` e `logout()`.



Base para Autenticação Real

Arquitetura pronta para evoluir para JWT, API e interceptors nas próximas aulas.

Checklist Final

Antes de avançar para a próxima aula, verifique se cada item está funcionando corretamente no seu projeto:

- ☐ Guard criado em `core/auth.guard.ts` com o tipo `CanActivateFn`
- ☐ Signal `usuarioLogado` criado e exportado de `core/auth.ts`
- ☐ Funções `login()` e `logout()` criadas e funcionando
- ☐ Rota `/carrinho` protegida com `canActivate: [authGuard]`
- ☐ Botões de Login e Logout exibidos no template do App
- ☐ Teste 1 passou: acesso bloqueado sem login
- ☐ Teste 2 passou: acesso permitido após login
- ☐ Teste 3 passou: acesso bloqueado novamente após logout



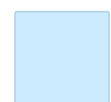
O Router não controla apenas navegação.

Ele controla **quem pode acessar** cada parte da aplicação.

Guarde esse conceito. Ele é a base de toda arquitetura de segurança em Angular.

Encerramento da Aula

Hoje você deu um passo importante na evolução do seu projeto Angular. A primeira camada de segurança está implementada, compreendida e testada.



O que você construiu

Um Guard funcional real, integrado ao Router e ao estado da aplicação — com testes manuais que confirmam o funcionamento.



O que vem a seguir

Nas próximas aulas, essa base será expandida com autenticação real, tokens JWT, interceptors e integração com API — tornando a aplicação pronta para produção.

✔ Você agora domina o conceito central de Guards no Angular. Continue praticando e nos vemos na próxima aula!

Dúvidas

?

